

Arquitetura e Organização de Computadores

Estrutura e Função do Processador: Pipeline

Rodrigo Calvo
rcalvo@uem.br

Departamento de Informática – DIN
Universidade Estadual de Maringá – UEM

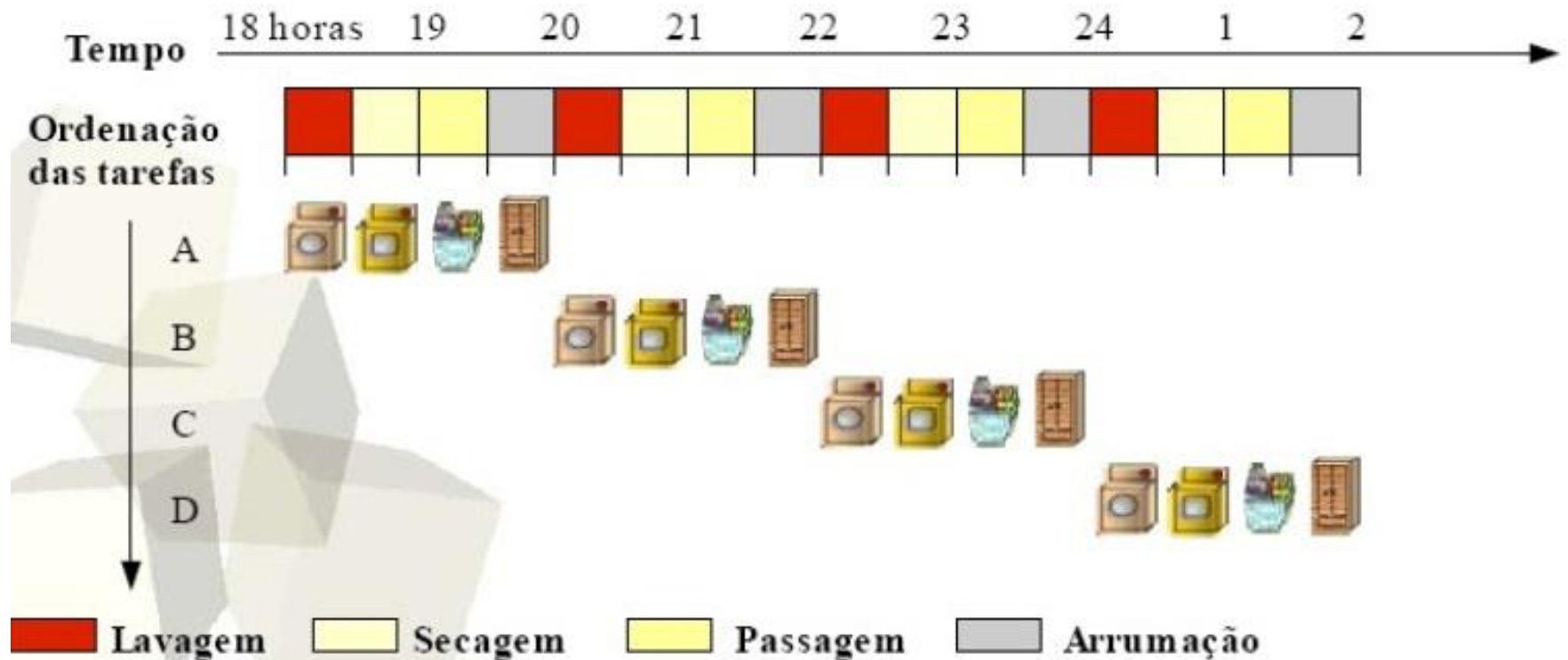
Pipelining

- Ciclo para execução de uma instrução:
 - Buscar instrução (FI – *Fetch Instruction*).
 - Decodificar instrução (DI – *Decode Instruction*).
 - Calcular operandos (CO – *Calculate Operands*).
 - Buscar operandos (FO – *Fetch Operands*).
 - Executar instruções (EI – *Execute Instruction*).
 - Escrever resultado (WO – *Write Operand*).
- Ideia: Sobrepor estas operações!

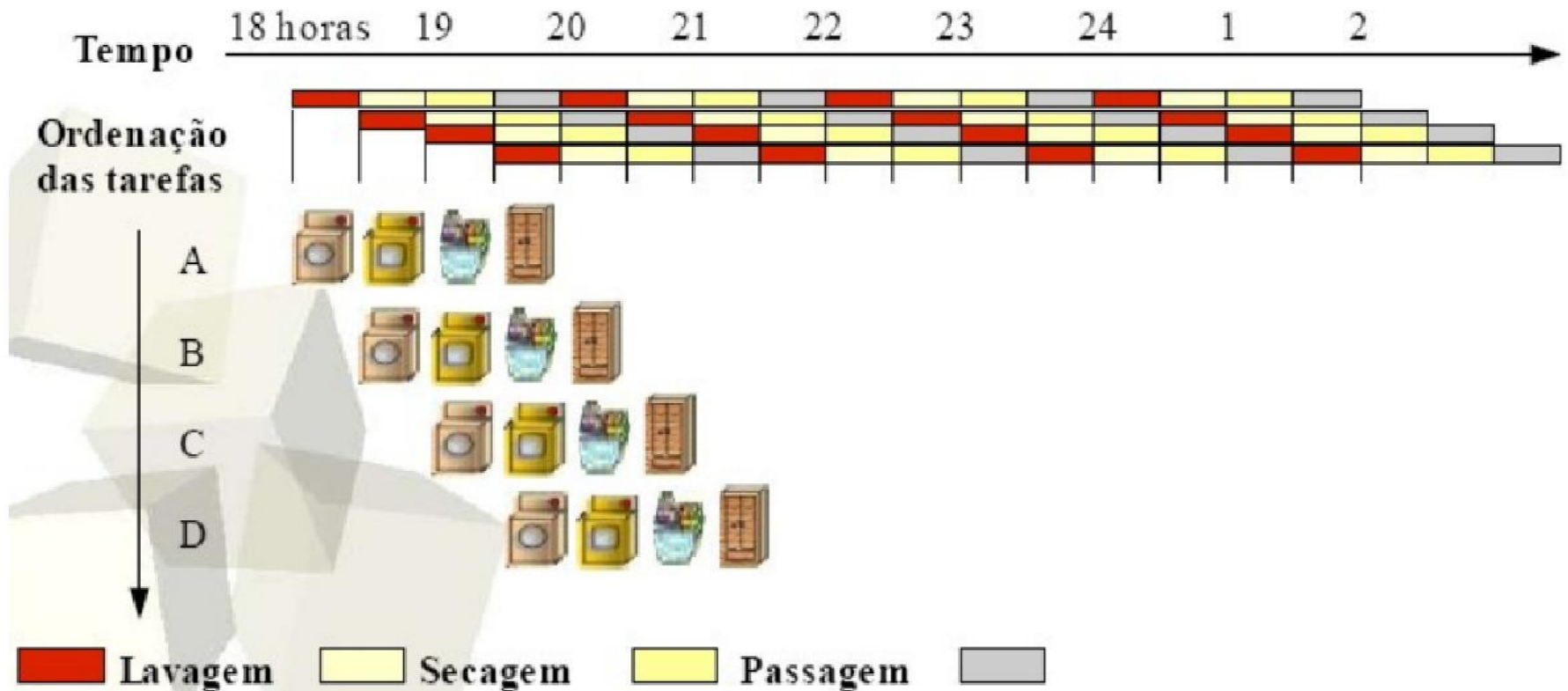
Pipelining

- Pipeline é uma técnica de implementação de processadores que permite a sobreposição temporal das diversas fases de execução das instruções.
- Semelhante a uma linha de montagem industrial, onde produtos em vários estágios podem ser trabalhados simultaneamente.
- Aumenta o número de instruções executadas simultaneamente e a taxa de instruções iniciadas e terminadas por unidade de tempo.
 - Aumenta o chamado de *throughput*.
- O pipeline não reduz o tempo gasto para completar cada instrução individualmente.

Pipelining: analogia sem Pipeline



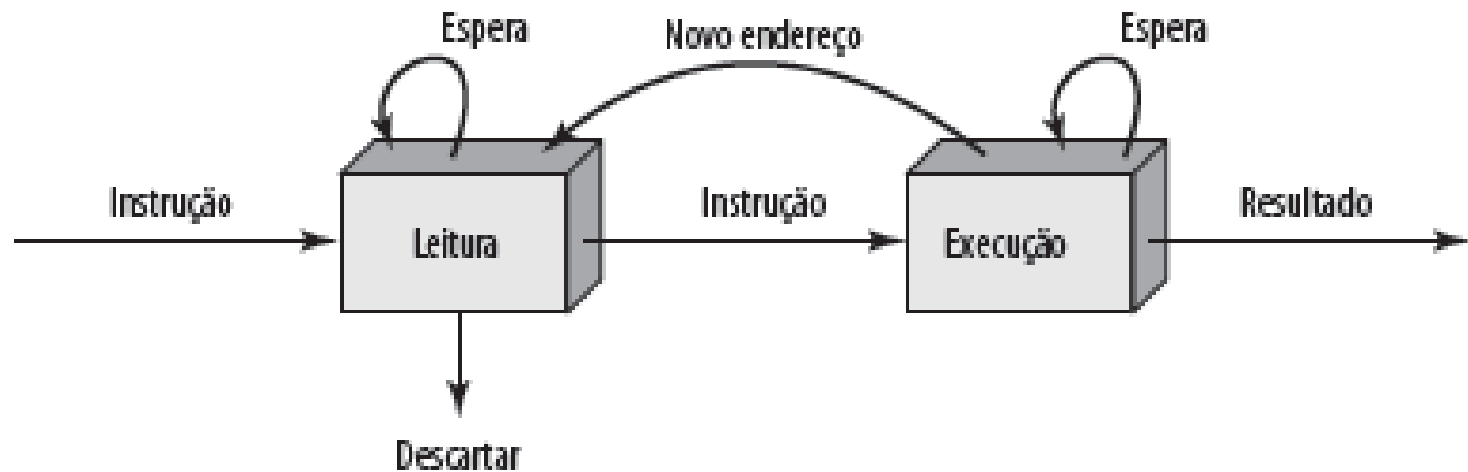
Pipelining: analogia com Pipeline



Pipeline de instrução de dois estágios



(a) Visão simplificada



(b) Visão expandida

Pipeline: Diagrama de tempo

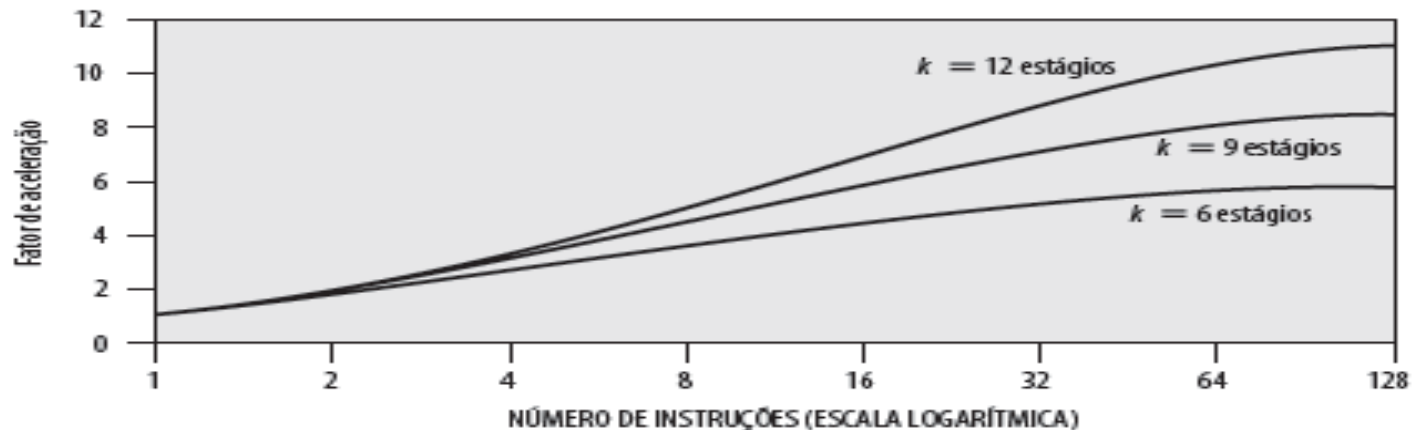
Tempo →

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instrução 1	FI	DI	CO	FO	EI	WO								
Instrução 2		FI	DI	CO	FO	EI	WO							
Instrução 3			FI	DI	CO	FO	EI	WO						
Instrução 4				FI	DI	CO	FO	EI	WO					
Instrução 5					FI	DI	CO	FO	EI	WO				
Instrução 6						FI	DI	CO	FO	EI	WO			
Instrução 7							FI	DI	CO	FO	EI	WO		
Instrução 8								FI	DI	CO	FO	EI	WO	
Instrução 9									FI	DI	CO	FO	EI	WO

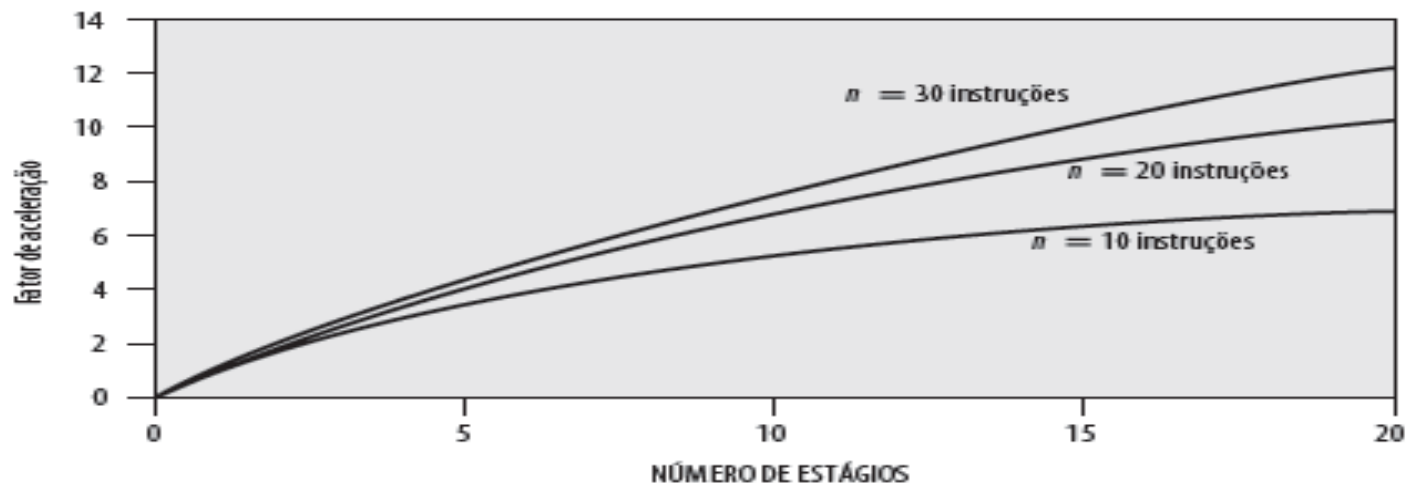
Efetivo de desvio condicional

	Tempo →							← Penalidade por desvio						
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instrução 1	FI	DI	CO	FO	EI	WO								
Instrução 2		FI	DI	CO	FO	EI	WO							
Instrução 3			FI	DI	CO	FO	EI	WO						
Instrução 4				FI	DI	CO	FO							
Instrução 5					FI	DI	CO							
Instrução 6						FI	DI							
Instrução 7							FI							
Instrução 15								FI	DI	CO	FO	EI	WO	
Instrução 16									FI	DI	CO	FO	EI	WO

Fatores de aceleração com pipeline de instrução



(a)



(b)

Hazards do pipeline

- Pipeline, ou alguma parte do pipeline, precisa parar.
- Também conhecida como bolha no pipeline.
- Tipos de hazards:
 - Recursos
 - Dados
 - Controle

Hazards de recursos

- Duas (ou mais) instruções no pipeline precisam do mesmo recurso.
- Também chamado hazard estrutural.

Hazards de recursos

- Exemplo: Considere pipeline simplificado em 5 estágios.
 - Cada estágio usa um ciclo de clock.
- No caso ideal, cada nova instrução entra no pipeline a cada ciclo de clock.
- Suponha que a memória principal tenha única porta.
- Considere buscas de instrução, leituras e escritas de dados uma por vez.
- Desta forma, leitura ou escrita de operando não podem ser realizadas em paralelo com busca de instrução.

Hazards de recursos

- Estágio de busca de instrução fica ocioso por um ciclo buscando I3.

	Ciclo de clock								
	1	2	3	4	5	6	7	8	9
Instrução	I1	FI	DI	FO	EI	WO			
	I2		FI	DI	FO	EI	WO		
	I3			FI	DI	FO	EI	WO	
	I4				FI	DI	FO	EI	WO

(a) Pipeline de cinco estágios, caso ideal

- Uma solução: Aumentar recursos disponíveis.
 - Neste caso, aumentar as portas de acesso a memória principal.

		Ciclo de clock								
		1	2	3	4	5	6	7	8	9
Instrução	I1	FI	DI	FO	EI	WO				
	I2		FI	DI	FO	EI	WO			
	I3			Ócioso	FI	DI	FO	EI	WO	
	I4					FI	DI	FO	EI	WO

(b) Operando de origem de I1 na memória

Hazards de dados

- Conflito no acesso de um local de operando.
- Duas instruções a serem executadas em sequência.
- Ambas acessam uma região de memória em particular ou operando do registrador.
- Se na sequência estrita, não ocorre problema.
- Se em um pipeline, valor do operando poderia ser atualizado para produzir resultado diferente da execução sequencial estrita.

Hazards de dados

- Exemplo: Sequência de instruções de máquina do x86:

ADD eax, ebx /* $eax = eax + ebx$

SUB ecx, eax /* $ecx = ecx - eax$

- Instrução ADD não atualiza EAX até o fim do estágio 5, no ciclo de clock 5.
- Instrução SUB precisa do valor no início do seu estágio 3, no ciclo de clock 4.
- Pipeline precisa parar por dois ciclos de clock.**
 - Como fazer para “parar o pipeline” por alguns estágios?
- Sem hardware especial e algoritmos de impedimento específicos, resulta em uso ineficaz do pipeline.

Exemplo de Hazards de dados

		Ciclo de clock									
		1	2	3	4	5	6	7	8	9	10
ADD EAX, EBX		FI	DI	FO	EI	EO					
SUB ECX, EAX			FI	DI	Ócioso		FO	EI	WO		
I3				FI			DI	FO	EI	WO	
I4							FI	DI	FO	EI	WO

Tipos de Hazards de dados

- Leitura após escrita (RAW), ou dependência verdadeira:
 - Uma instrução modifica um registrador ou local de memória.
 - Instrução seguinte lê dados nesse local.
 - Hazard se leitura ocorre antes do término da escrita.
- Exemplo:

ADD R3, R2, R1 ; $R3 = R2 + R1$

SUB R4, R3, 1 ; $R4 = R3 - 1$

Tipos de Hazards de dados

- Escrita após leitura (WAR), ou antidependência:
 - Uma instrução lê um registrador ou local da memória.
 - Instrução seguinte escreve no local.
 - Hazard se escrita termina antes que ocorra a leitura.
- Exemplo:

ADD R3, R2, R1 ; $R3 = R2 + R1$

SUB R4, R3, 1 ; $R4 = R3 - 1$

Tipos de Hazards de dados

- Escrita após escrita (WAW), ou dependência de saída:
 - Duas instruções escrevem no mesmo local.
 - Hazard se a escrita ocorre na ordem contrária à sequência intencionada.

- Exemplo:

ADD R3, R2, R1 ; $R3 = R2 + R1$

SUB R4, R3, 1 ; $R4 = R3 - 1$

ADD R3, R2, R5 ; $R3 = R2 + R5$

Técnicas para resolver Hazards de dados

- Reordenação de Código – Usa-se um compilador “inteligente” que reordena o código em Assembly para evitar esses tipos de dependência.
- Inserção de Bolhas – Inserimos uma bolha no pipeline para atrasar a execução da instrução, ganhando tempo para a instrução posterior. Uso de instruções NOP (*No Operation*).
 - Última opção.

Hazards de controle

- Também conhecido como hazard de desvio.
- Pipeline toma decisão errada sobre previsão de desvio.
- Traz para o pipeline instruções que precisam ser descartadas subsequentemente.
- Formas de lidar com desvios:
 - Múltiplos fluxos.
 - Busca antecipada do alvo do desvio.
 - Buffer de laço de repetição.
 - Previsão de desvio (Estática e Dinâmica).
 - Desvio atrasado.

Fluxos múltiplos

- Têm dois pipelines.
- Busca antecipada de cada desvio em um pipeline separado.
- Após desvio ser descoberto, usa-se pipeline apropriado.
- Problema:
 - Ocasiona disputa por barramento e registradores.
 - Ex.: Os dois lados querem modificar o mesmo registrador.

Busca antecipada do alvo de desvio

- Alvo do desvio é buscado antecipadamente além das instruções após o desvio.
- Mantém alvo até que o desvio seja executado.
- Caso o desvio seja tomado, a instrução alvo do desvio é inserida no pipeline.

Buffer de laço de repetição

- Buffer = Memória muito rápida.
- Mantido pelo estágio de busca do pipeline.
- Guardar n instruções buscadas mais recentemente, em sequência.
- Verificar buffer antes de buscar da memória.
- Geralmente é usado em conjunto com a busca antecipada.
- Muito bom para laços ou saltos pequenos.

Previsão de desvio

- Previsão nunca tomada:
 - Assume que salto não acontecerá.
 - Sempre busca próxima instrução.
- Previsão sempre tomada:
 - Assume que salto acontecerá.
 - Sempre busca instrução alvo.
- Abordagens simples e estáticas.

Previsão de desvio

- Previsão por *opcode*:
 - Algumas instruções são mais prováveis de resultar em um salto do que outras (Estatisticamente).
 - Pode chegar até 75% de sucesso.
 - Abordagem simples e estática também.

Previsão de desvio

- Previsão por chave tomada/não tomada:
 - Baseada no histórico do desvio.
 - Um ou mais bits podem ser associados com cada instrução de desvio condicional que reflete o seu histórico recente.
 - Geralmente utiliza-se 1 ou 2 bits.
 - 1 Bit: Sabe-se apenas sobre a última execução do desvio.
 - 2 Bits: Sabe-se sobre as duas últimas execuções do desvio.
 - Boa para laços (Executa várias vezes o mesmo desvio).
 - Abordagem dinâmica.

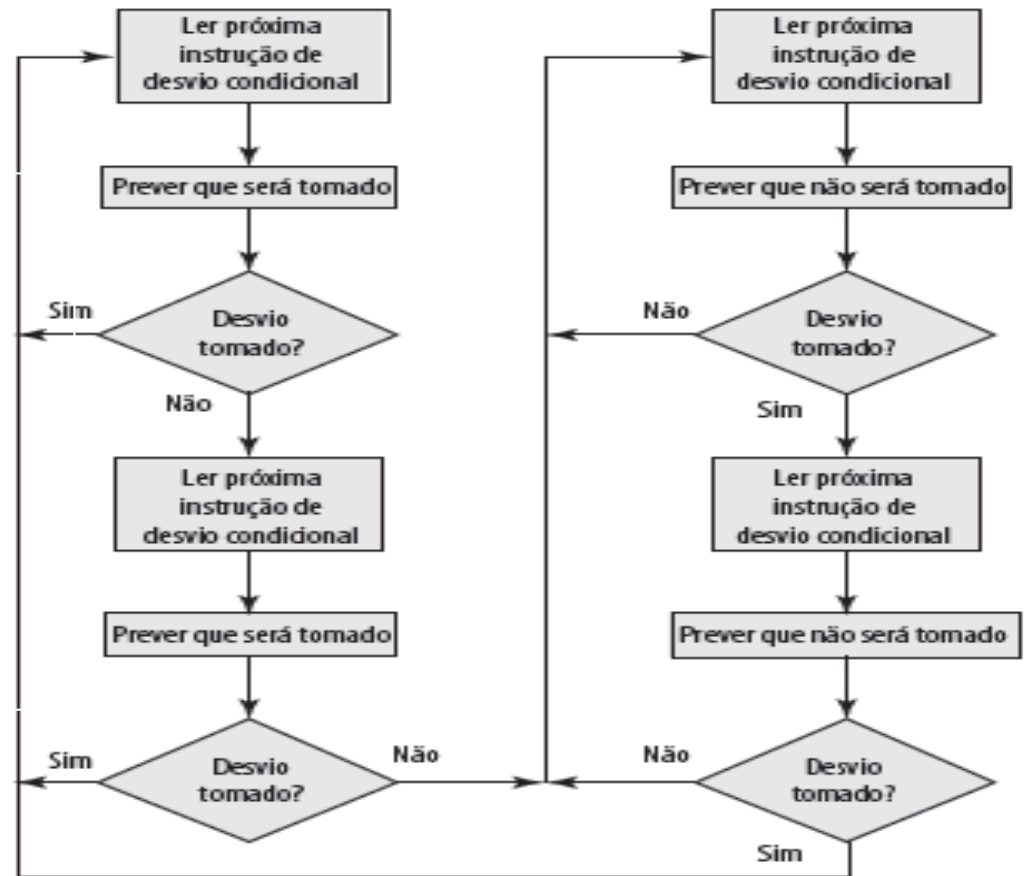
Fluxograma de chave tomada/não-tomada (2 bits)

00 – Nas duas últimas execuções não foi tomado.

01 – Na última foi tomado e na penúltima não.

10 – Na última não foi tomado e na penúltima foi.

11 – Nas duas últimas execuções foram tomados.



Previsão de desvio

- Tabela de histórico de desvio:
 - A tabela é uma pequena memória associada ao estágio de busca.
 - Usa-se também o comportamento de outros desvios para fazer uma predição, ao invés de apenas aquele que está tentando predizer.
 - Guarda-se o histórico global dos desvios mais recentes.
 - Quando um desvio condicional é encontrado, tenta-se prever se ele será ou não tomado de acordo com a tabela de histórico.

Desvio atrasado

- Não salta até que realmente seja preciso
- Reorganiza as instruções

Bibliografia

- Sttallings, W. **Arquitetura e organização de computadores: projeto para o desempenho**. 8. ed. Prentice Hall, 2009.
- Delgado, J.; Ribeiro, C. **Arquitetura de Computadores**. 2 ed. LTC, 2009.
- Patterson, D. A. ; Hennessy, J.L. **Organização e projeto de computadores – a interface hardware software**. 3. ed. Editora Campus, 2005.